

```
import numpy as np

# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])

# Addition
print("Addition (A + B):")
print(matrix_a + matrix_b)
# Subtraction
print("Subtraction (A - B):")
print(matrix_a - matrix_b)
# Multiplication (element-wise)
print("Element-wise Multiplication (A * B):")
print(matrix_a * matrix_b)
# Matrix multiplication (dot product)
print("A dot B:")
print(np.dot(matrix_a, matrix_b))
# Transpose
print("Transpose of A:")
print(matrix_a.T)
```

```
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
a = np.hstack((arr1, arr2))
print("Horizontal Stack:")
print(a)

# Perform vertical stacking (vstack)
b = np.vstack((arr1, arr2))
print("Vertical Stack:")
print(b)
```

```
import numpy as np
```

```
# Take user input for the start, stop, and step of  
the sequence
```

```
start = int(input())
```

```
stop = int(input())
```

```
step = int(input())
```

```
# Generate the sequence using np.arange()
```

```
a = np.arange(start, stop, step)
```

```
# Print the generated sequence
```

```
print(a)
```

```
import numpy as np

def array_operations(A, B):

    # Convert A and B to NumPy arrays
    A = np.array(A)
    B = np.array(B)
    # Arithmetic Operations
    sum_result = A + B
    diff_result = A - B
    prod_result = A * B

    # Statistical Operations
    mean_A = np.mean(A)
    median_A = np.median(A)
    std_dev_A = np.std(A)

    # Bitwise Operations
    and_result = A & B
    or_result = A | B
    xor_result = A ^ B

    # Output results with one space between each element
    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
    print("Element-wise Product:", ' '.join(map(str, prod_result)))

    print(f"Mean of A: {mean_A}")
    print(f"Median of A: {median_A}")
    print(f"Standard Deviation of A: {std_dev_A}")

    print("Bitwise AND:", ' '.join(map(str, and_result)))
    print("Bitwise OR:", ' '.join(map(str, or_result)))
    print("Bitwise XOR:", ' '.join(map(str, xor_result)))

A = list(map(int, input().split())) # Elements of array A
B = list(map(int, input().split())) # Elements of array B
array_operations(A, B)
```

```
import numpy as np

inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array = original_array.view()

# Create a copy
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

```
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split()))))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))

# Find indices where value matches in array1
indices = np.where(array1 == search_value)
print(indices[0])
# Count occurrences in array1
count = np.count_nonzero(array1 == count_value)
print(count)
# Broadcasting addition
broadcast_result = array1 + broadcast_value
print(broadcast_result)
# Sort the first array
sorted_array = np.sort(array1)
print(sorted_array)
```

```

import numpy as np

a = np.loadtxt("Sample.csv", delimiter=";", skiprows=1)
# 1. Print all student details
print("All student Details:\n", a)

# 2. print total students
print("Total Students:", a.shape[0])

# 3. Print all student Roll numbers
print("All Student Roll Nos", a[:,0])

# 4. Print subject 1 marks
print("Subject 1 Marks", a[:,1])

# 5. print minimum marks of Subject 2
print("Min marks in Subject 2", np.min(a[:,2]))

# 6. print maximum marks of Subject 3
print("Max marks in Subject 3", np.max(a[:,3]))

# 7. Print All subject marks
print("All subject marks:", a[:,1:])

# 8. print Total marks of students
print("Total Marks", np.sum(a[:,1:], axis=1))

# 9. print average marks of each student
avg = np.mean(a[:,1:], axis=1)
print(np.round(avg, 1))

# 10. print average marks of each subject
print("Average Marks of each subject", np.mean(a[:,1:], axis=0))

import numpy as np

# --- Slide 1 ---

# 11. print average marks of S1 and S2
print("Average Marks of S1 and S2", np.mean(a[:, 1:3], axis=0))

# 12. print average marks of S1 and S3
print("Average Marks of S1 and S3", np.mean(a[:, [1, 3]], axis=0))

# --- Slide 2 ---

# 13. print Roll number who got maximum marks in Subject 3
j = np.argmax(a[:, 3])
print("Roll no who got maximum marks in Subject 3", a[j, 0])

# 14. print Roll number who got minimum marks in Subject 2
j = np.argmin(a[:, 2])
print("Roll no who got minimum marks in Subject 2", a[j, 0])

# 15. print Roll number who got 24 marks in Subject 2
i = np.where(a[:, 2] == 24)
print("Roll no who got 24 marks in Subject 2", a[i, 0])

# 16. print count of students who got marks in Subject 1 < 40
print("Count of students who got marks in Subject 1 < 40", np.sum(a[:, 1] < 40))

# 17. print count of students who got marks in Subject 2 > 90
print("Count of students who got marks in Subject 2 > 90:", np.sum(a[:, 2] > 90))

# 18. print count of students in each subject who got marks >= 90
print("Count of students in each subject who got marks >= 90:", np.sum(a[:, 1:] >= 90, axis=0))

# 19. print count of subjects in which each student got marks >= 90
print("Roll no:", a[:, 0])
print("Count of subjects in which student got marks >= 90:", np.sum(a[:, 1:] >= 90, axis=1))

# 20. Print S1 marks in ascending order
print(np.sort(a[:, 1]))

# 21. Print S1 marks >= 50 and <= 90
print(a[(a[:, 1] >= 50) & (a[:, 1] <= 90)])

# 22. Print the index position of marks 79
print(a)
print((np.where(a == 79)[0]),)

```